BLOCKCHAIN LABORATORY

Experiment No. 10

Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Name: Nikhil Sunchu

Roll No.: 31

Class/section: D20A

Date: 25 October 2026

1. Aim

To implement a Hyperledger Fabric network and perform asset management operations using chaincode.

2. Theory

2.1. 1. Hyperledger Fabric

Hyperledger Fabric is a permissioned blockchain platform designed for enterprise applications. A Fabric network consists of peer nodes, an ordering service, identities, and channels. Peers host ledgers and execute chaincode, while the ordering service orders endorsed transactions into blocks for distribution to peers.

2.2. 2. Peers, Ordering Service, and Channels

Peers maintain the ledger and participate in transaction endorsement and validation. The ordering service establishes the order of transactions and delivers blocks to channel members. A channel provides a private logical ledger shared by the organizations that have joined it. In this experiment, a local test network was started with two peer organizations and an ordering service, and the channel `nikhilsunchu` was created.

2.3. 3. Chaincode

Chaincode is the application logic deployed on a Fabric network. It defines the operations that clients can invoke to read or modify ledger state. The sample `asset-transfer-basic` chaincode was deployed under the name `basic`. Its asset functions support creation, reading, updating, transfer of ownership, deletion, and retrieval of assets.

2.4. 4. Transactions and Ledger State

A query reads current ledger state without submitting a state-changing transaction. An invoke submits a transaction proposal to peers and the ordering service. Once validated and committed, the transaction updates the ledger state. In this experiment, successful invokes returned status `200`; querying the deleted asset returned an asset-not-found response, confirming the deletion.

3. Tools and Environment Used

- Linux terminal and Docker.
- Docker Compose.

- Hyperledger Fabric test network, version 2.5.16.
- Fabric command-line binaries, including `peer`.
- Fabric sample chaincode: `asset-transfer-basic` (Go).
- Channel name: `nikhilsunchu`.
- Chaincode name: `basic`, version 1.0.

4. Tasks Performed

4.1. Task 1 – Verify the Environment and Install Fabric

Verified that Docker and Docker Compose were available. Downloaded the official Fabric samples, binaries, and Docker images. The required peer, orderer, and base images for Fabric 2.5.16 were present. Go was installed to package the Go-based chaincode.

```
Blockchain/Exp10 ❯ ls                                                    main   took 59m18s
fabric-samples  install-fabric.sh
Blockchain/Exp10 ❯ docker images                                         main
                                                                  1 Info →  U  In Use
IMAGE                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
cloudera/quickstart:latest                     22500f6320ac         11GB          4.44GB   U
elasticsearch:9.3.3                            1d9ddbe28380       2.15GB           719MB
ghcr.io/hyperledger/fabric-baseos:2.5.16       109b4a15b282        257MB          83.2MB
ghcr.io/hyperledger/fabric-ca:1.5.17           453a0a727e82        381MB           123MB
ghcr.io/hyperledger/fabric-orderer:2.5.16      1d918fc7e1e0        182MB          50.7MB
ghcr.io/hyperledger/fabric-peer:2.5.16         58979b65744b        232MB          66.8MB
ghcr.io/zammad/zammad:7.0.1-0019               f50e204dc85b       1.45GB           320MB
grafana/grafana:latest                         5dad0df181cb       1.47GB           352MB
hyperledger/fabric-baseos:2.5                  109b4a15b282        257MB          83.2MB
hyperledger/fabric-baseos:2.5.16               109b4a15b282        257MB          83.2MB
hyperledger/fabric-baseos:latest               109b4a15b282        257MB          83.2MB
hyperledger/fabric-ca:1.5                      453a0a727e82        381MB           123MB
hyperledger/fabric-ca:1.5.17                   453a0a727e82        381MB           123MB
hyperledger/fabric-ca:latest                   453a0a727e82        381MB           123MB
hyperledger/fabric-orderer:2.5                 1d918fc7e1e0        182MB          50.7MB
hyperledger/fabric-orderer:2.5.16              1d918fc7e1e0        182MB          50.7MB
hyperledger/fabric-orderer:latest              1d918fc7e1e0        182MB          50.7MB
hyperledger/fabric-peer:2.5                    58979b65744b        232MB          66.8MB
hyperledger/fabric-peer:2.5.16                 58979b65744b        232MB          66.8MB
hyperledger/fabric-peer:latest                 58979b65744b        232MB          66.8MB
mariadb:10.11                                  407fccb51e71        466MB           111MB
mariadb:11                                     78a5047d3ba3        465MB           111MB
mediawiki:latest                               7245a794192d       1.53GB           330MB
memcached:1.6.41-alpine                        65c80b311a96       20.1MB          6.08MB
metabase/metabase:latest                       2a9b72c0c049       1.76GB           769MB
migrate/migrate:v4.18.1                        f21c436af23c       82.7MB            20MB
mysql:8.0                                      d0304ed9fdb6        1.1GB           249MB
postgres:16                                    71e27bf60b70        641MB           166MB
postgres:17.9-alpine                           c7526c0f6c3f        399MB           112MB
prom/prometheus:latest                         69f524141883        593MB           156MB
redis:7-alpine                                 6ab0b6e73817       57.8MB          16.8MB
redis:8.6.2-alpine                             c5e375abb885        134MB            35MB
snipe/snipe-it:latest                          e9458d80cf40       1.37GB           301MB
timescale/timescaledb:latest-pg16              51ac20ec2956       1.94GB           487MB
zabbix/zabbix-server-mysql:latest              4cddcdaff354        145MB          36.8MB
zabbix/zabbix-web-nginx-mysql:latest           5e954e507448        246MB          60.4MB
Blockchain/Exp10 ❯
```

Figure 1: Hyperledger Fabric Docker images available locally

4.2. Task 2 – Start the Network and Create a Channel

Started the Fabric test network and created the channel `nikhilsunchu`. The network included `peer0.org1.example.com`, `peer0.org2.example.com`, and `orderer.example.com`. Org2 was joined to the channel.

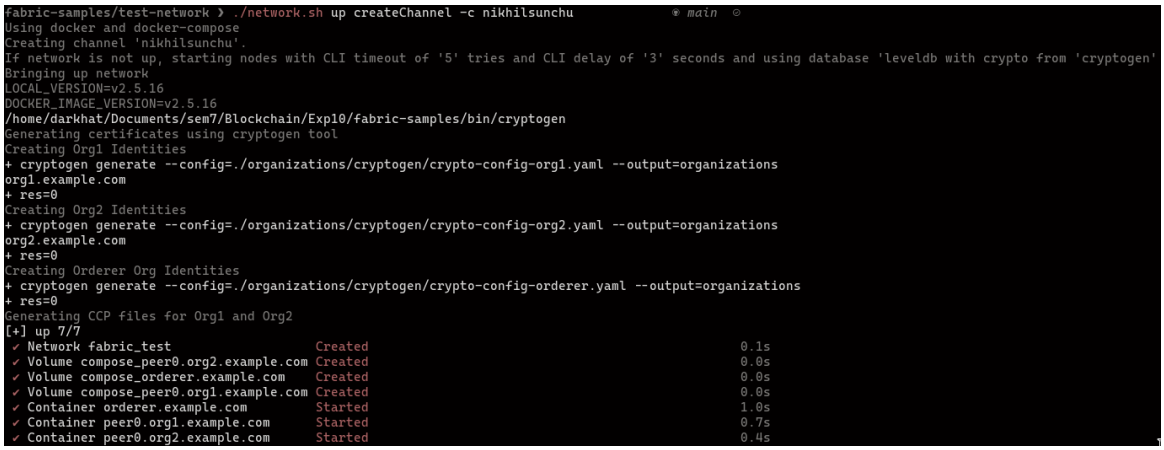


Figure 2: Starting the Fabric test network

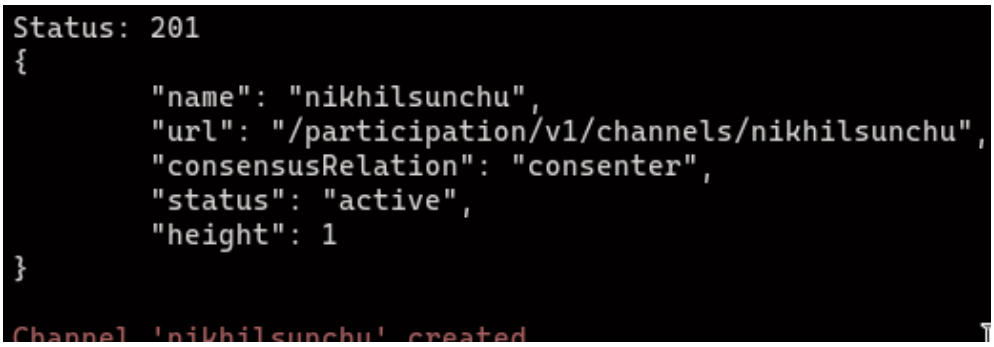


Figure 3: Channel creation response for `nikhilsunchu`



Figure 4: Org2 peer joined the channel

4.3. Task 3 – Deploy and Verify Chaincode

Deployed the Go-based `asset-transfer-basic` sample as chaincode `basic`. After resolving the missing Go compiler and restarting the Fabric containers, queried the committed definition. The output confirmed version 1.0, sequence 1, and approvals from both `Org1MSP` and `Org2MSP`.

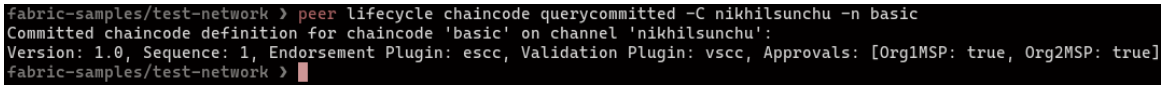


Figure 5: Committed chaincode definition and organization approvals

4.4. Task 4 – Create and Read an Asset

Invoked `CreateAsset` to create `asset10` with color `blue`, size 5, owner `Nikhil`, and appraised value 500. Used `ReadAsset` to query the asset and verify that its fields were stored on the ledger.

```
CreateAsset("asset10", "blue", "5", "Nikhil", "500")
ReadAsset("asset10")
```

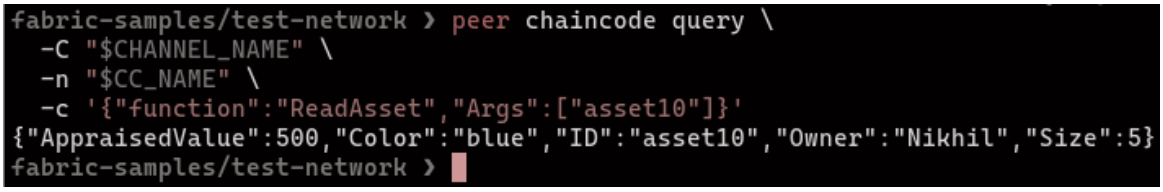


Figure 6: Query result for the asset

4.5. Task 5 – Update Asset Attributes

Invoked `UpdateAsset` to change the asset color to `green`, size to 10, and appraised value to 750. The owner remained `Nikhil`. A subsequent query confirmed the updated values.



Figure 7: Successful asset update and verification

4.6. Task 6 – Transfer Asset Ownership

Invoked `TransferAsset` to transfer `asset10` to `Rahul`. The transaction returned status 200. A subsequent `ReadAsset` query confirmed that the owner had changed to `Rahul` while the other fields remained unchanged.



Figure 8: Successful ownership transfer and updated asset state

4.7. Task 7 – Delete an Asset and Verify

Created a separate test asset named `asset11` and invoked `DeleteAsset`. The delete transaction returned status 200. A subsequent `ReadAsset` query returned the message that `asset11` does not exist, confirming deletion.



Figure 9: Asset deletion and expected asset-not-found response

4.8. Task 8 – Verify the Final Ledger State

Queried `asset10` and then invoked `GetAllAssets`. The final output showed `asset10` with owner `Rahul`, color `green`, size 10, and appraised value 750. The asset list contained `asset10`; `asset11` was absent.

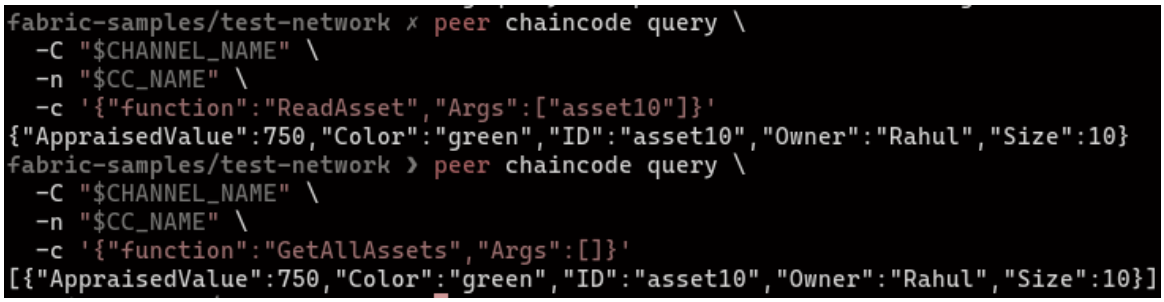


Figure 10: Final asset state and output of `GetAllAssets`

5. Result

A Hyperledger Fabric test network was established with two peer organizations and an ordering service. The channel `nikhilsunchu` was created, and chaincode `basic` was deployed and committed with approvals from both organizations. Asset creation, querying, updating, ownership transfer, and deletion were performed successfully. The final ledger query confirmed that `asset10` remained with owner `Rahul`, color `green`, size 10, and appraised value 750, while `asset11` had been deleted.

6. Conclusion

The experiment provided practical experience in configuring a Hyperledger Fabric network, creating a channel, deploying chaincode, and interacting with ledger assets. It demonstrated how chaincode implements asset-management operations and how transaction responses and subsequent queries can be used to verify changes to the ledger state.